
Integration Manual

for MPC560XB WDG Driver

Document Number: IM13WDGASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	7
2.2	Overview.....	8
2.3	About this Manual.....	8
2.4	Acronyms and Definitions.....	8
2.5	Reference List.....	9
Chapter 3		
Building the Driver		
3.1	Build Options.....	11
3.1.1	CW Compiler/Linker/Assembler Options.....	11
3.1.2	DIAB Compiler/Linker/Assembler Options.....	13
3.1.3	GHS Compiler/Linker/Assembler Options.....	15
3.1.4	HT Compiler/Linker/Assembler Options.....	16
3.2	Files required for Compilation.....	17
3.3	Setting up the Plug-ins.....	19
Chapter 4		
Function calls to module		
4.1	Function Calls during Start-up.....	21
4.2	Function Calls during Shutdown.....	21
4.3	Function Calls during Wake-up.....	21
Chapter 5		
Module requirements		
5.1	Exclusive areas to be defined in BSW scheduler.....	23
5.2	Peripheral Hardware Requirements.....	23
5.3	ISR to configure within OS – dependencies.....	23

Section number	Title	Page
5.4	ISR Macro.....	24
5.5	Other AUTOSAR modules - dependencies.....	24
Chapter 6		
Main API Requirements		
6.1	Main functions calls within BSW scheduler.....	27
6.2	API Requirements.....	27
6.3	Calls to Notification Functions, Callbacks, Callouts.....	27
6.3.1	Call-back Notifications: User Notification:.....	27
6.3.3	User Notification:.....	28
Chapter 7		
Memory Allocation		
7.1	Sections to be defined in MemMap.h.....	29
7.2	Linker command file.....	31
Chapter 8		
Configuration parameters considerations		
8.1	Configuration parameters considerations.....	33
Chapter 9		
Integration Steps		
Chapter 10		
ISR Reference		
10.1	Software specification.....	39
10.1.1	Define Reference.....	39
10.1.2	Macros Reference.....	39
10.1.3	Enum Reference.....	39
10.1.4	Function Reference.....	39
10.1.4.1	Function Wdg_Isr.....	39
10.1.5	Structs Reference.....	40
10.1.6	Types Reference.....	40
10.1.7	Variables Reference.....	40

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	30-Jan-2012	Salvatore Nappi	Initial release for MPC560XB
1.1	10-May-2013	Jayant Shekhar	Bolero ASR 4.0 RTM 1.0.0 Release
1.2	02-May-2014	Salvatore Nappi	Added multinstance reference and callback specification



Chapter 2

Introduction

This integration manual describes the integration requirements for WDG Driver for MPC560XB microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	---

All of the above microcontroller devices are collectively named as MPC560XB .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface

Table continues on the next page...

Table 2-2. Acronyms and Definitions (continued)

Term	Definition
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
LT Variant	Link Time Variant
OS	Operating System
ISR	Interrupt Service Routine

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003WDG Driver Software Specification Document.	4.0 Rev0003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.1, May 2012
4	5606B Reference Manual	Rev. 1, 12 Jul 2011
5	5602D Reference Manual	Rev. 4.1, May 2012
6	Bolero 1.5M cut 2.2 Errata Sheet Reference Manual	MPC5607B_1M03Y - Rev. 30Aug2012
7	Bolero 512k cut 2.3 Errata Sheet Reference Manual	MPC5604B_2M27V - Rev 17 MAY 2012
8	Bolero 256k cut 1.1 Errata Sheet Reference Manual	MPC5602D_1M18Y - Rev 17 MAY 2012

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar WDG driver for Freescale Semiconductor MPC560XB . It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The WDG driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.1.1, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note

The TS_T2D13M10I0R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-2. Assembler Options (continued)

Option	Description
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as;,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'. Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-L	Add path to the list of paths that ld will search for archive libraries and ld control scripts.
-T	The -T option specifies the name of your own script file sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the WDG driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

WDG Files

- ..\Wdg_TS_T2D13M10I0R0\src\Wdg_Channel.c
- ..\Wdg_TS_T2D13M10I0R0\src\Wdg_LLD.c
- ..\Wdg_TS_T2D13M10I0R0\src\Wdg_Irq.c
- ..\Wdg_TS_T2D13M10I0R0\src\Wdg_Swt_LLD.c
- ..\Wdg_TS_T2D13M10I0R0\include\Wdg.h
- ..\Wdg_TS_T2D13M10I0R0\include\Wdg_LLD.h
- ..\Wdg_TS_T2D13M10I0R0\include\Reg_eSys_SWT.h
- ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Swt_LLD.h
- ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Cbk.h
- ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Channel.h

WDG Generated Files

- Wdg_43_Instance0_PBCfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation
- Wdg_43_Instance0_Lcfg.c (For LT Variant) - This file should be generated by the user using a configuration tool for compilation
- Wdg_43_Instance0_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation
- Wdg_CfgExt.c - This file should be generated by the user using a configuration tool for compilation
- Wdg_Cfg.h - This file should be generated by the user using a configuration tool for compilation

Files from Base common folder

- ..\Base_TS_T2D13M10I0R0\autosar\Base.epd
- ..\Base_TS_T2D13M10I0R0\config\Base.xdm
- ..\Base_TS_T2D13M10I0R0\generate_PC\include\modules.h
- ..\Base_TS_T2D13M10I0R0\include\Can_GeneralTypes.h
- ..\Base_TS_T2D13M10I0R0\include\Cer.h
- ..\Base_TS_T2D13M10I0R0\include\Compiler.h
- ..\Base_TS_T2D13M10I0R0\include\Compiler_Cfg.h
- ..\Base_TS_T2D13M10I0R0\include\Reg_eSys.h
- ..\Base_TS_T2D13M10I0R0\include\ComStack_Cfg.h
- ..\Base_TS_T2D13M10I0R0\include\ComStack_Types.h
- ..\Base_TS_T2D13M10I0R0\include\Mcal.h
- ..\Base_TS_T2D13M10I0R0\include\MemMap.h
- ..\Base_TS_T2D13M10I0R0\include\Platform_Types.h
- ..\Base_TS_T2D13M10I0R0\include\Reg_Macros.h
- ..\Base_TS_T2D13M10I0R0\include\Soc_Ips.h
- ..\Base_TS_T2D13M10I0R0\include\Std_Types.h

Files from Dem folder:

- ..\Dem_ TS_T2D13M10I0R0 \include\Dem.h
- ..\Dem_ TS_T2D13M10I0R0 \include\Dem_IntErrId.h
- ..\Dem_ TS_T2D13M10I0R0 \include\Dem_Types.h
- ..\Dem_ TS_T2D13M10I0R0 \src\Dem.c

Files from Det folder:

- ..\Det_ TS_T2D13M10I0R0 \include\Det.h
- ..\Det_ TS_T2D13M10I0R0 \src\Det.c

Files from EcuM folder:

- ..\EcuM_ TS_T2D13M10I0R0 \autosar\EcuM.epd
- ..\EcuM_ TS_T2D13M10I0R0 \config\EcuM.xdm
- ..\EcuM_ TS_T2D13M10I0R0 \generate_PC\include\EcuM_Cfg.h
- ..\EcuM_ TS_T2D13M10I0R0 \include\EcuM.h
- ..\EcuM_ TS_T2D13M10I0R0 \include\EcuM_Cbk.h
- ..\EcuM_ TS_T2D13M10I0R0 \src\EcuM.c

Files from Resource folder:

- ..\Resource_ TS_T2D13M10I0R0 \autosar\Resource.epd
- ..\Resource_ TS_T2D13M10I0R0 \config\Resource.xdm

Files from Rte folder:

- ..\Rte_ TS_T2D13M10I0R0 \include\SchM_Wdg.h
- ..\Rte_ TS_T2D13M10I0R0 \src\SchM_Wdg.c

Files from Gpt folder:

- ..\Gpt_ TS_T2D13M10I0R0 \autosar\Gpt.epd
- ..\Gpt_ TS_T2D13M10I0R0 \config\Gpt.xdm

Files from Mcu folder:

- ..\Mcu_ TS_T2D13M10I0R0 \autosar\Mcu.epd
- ..\Mcu_ TS_T2D13M10I0R0 \config\Mcu.xdm

3.3 Setting up the Plug-ins

The WDG driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.0.0 (b130225-0637) or later.)

Location of various files inside the module folder:

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:
 - ..\ Wdg_ TS_T2D13M10I0R0 \config\Wdg.xdm
 - ..\ Gpt_ TS_T2D13M10I0R0 \config\Gpt.xdm

- ..\Mcu_TS_T2D13M10I0R0\config\Mcu.xdm
- ..\Base_TS_T2D13M10I0R0\config\Base.xdm
- ..\Dem_TS_T2D13M10I0R0\config\Dem.xdm
- ..\Resource_TS_T2D13M10I0R0\config\Resource.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - ..\Wdg_TS_T2D13M10I0R0\autosar\Wdg.epd
 - ..\Gpt_TS_T2D13M10I0R0\autosar\Gpt.epd
 - ..\Mcu_TS_T2D13M10I0R0\autosar\Mcu.epd
 - ..\Base_TS_T2D13M10I0R0\autosar\Base.epd
 - ..\Dem_TS_T2D13M10I0R0\autosar\Dem.epd
 - ..\Resource_TS_T2D13M10I0R0\autosar\Resource.epd
- Code Generation Templates for Pre-Compile time configuration parameters:
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_43_Instance0_Cfg.c
 - ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Cfg.h
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_CfgExt.c
- Code Generation Templates for Post-Build time configuration parameters:
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_43_Instance0_PBCfg.c
 - ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Cfg.h
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_CfgExt.c
- Code Generation Templates for Link-Time configuration parameters:
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_43_Instance0_Lcfg.c
 - ..\Wdg_TS_T2D13M10I0R0\include\Wdg_Cfg.h
 - ..\Wdg_TS_T2D13M10I0R0\src\Wdg_CfgExt.c

Steps to generate the configuration:

1. Copy the module folders Wdg_TS_T2D13M10I0R0 , Wdg If_TS_T2D13M10I0R0 , Base_TS_T2D13M10I0R0 , Dem_TS_T2D13M10I0R0 , Det_TS_T2D13M10I0R0 , Rte_TS_T2D13M10I0R0 , Resource_TS_T2D13M10I0R0 , EcuM_TS_T2D13M10I0R0 , Mcu_TS_T2D13M10I0R0 , Gpt_TS_T2D13M10I0R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

Chapter 4

Function calls to module

4.1 Function Calls during Start-up

WDG shall be initialized to SLOW_MODE during STARTUP1 phase of EcuM initialization. The API to be called for this is Wdg_Init(). ECU State Manager has to trigger the WDG during STARTUP1 phase. The duration of STARTUP1 shall be kept as short as possible such that triggering is not needed during this phase. If triggering is required during STARTUP1 phase it shall be handled by EcuM.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

5.1 Exclusive areas to be defined in BSW scheduler

None.

5.2 Peripheral Hardware Requirements

A Software watchdog timer (SWT) with programmable interrupt response is available in MPC560XB. There are three user-defined responses to a time-out:

- If a time-out occurs, the SWT generates an interrupt to the processor core.
- The first time-out generates an interrupt to the processor, and if not serviced, then a second time-out generates a system reset and sets the MRSR[SWTR] flag.
- If a time-out occurs, the SWT generates a system reset and sets the MRSR [SWTR] flag.

In addition to these three modes of operation, the watchdog timer also supports a windowed mode. In this mode, the service sequence must be performed in the last part of the time-out period defined by the window register. The window is open when the down counter is less than the value in the SWT_WN register. Outside of this window, service sequence writes are invalid accesses and generate a bus error or reset depending on the value of the SWT_CR.RIA

5.3 ISR to configure within OS – dependencies

The following ISR's are used by the WDT driver:

Table 5-1. Wdg ISR's For STM channel

ISR Name	Hardware interrupt vector
Wdg_Isr	28

5.4 ISR Macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

a. OS is not used - AUTOSAR_OS_NOT_USED is defined:

i. If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

ii. If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

Custom OS is used - AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

Other vendor's OS is used - AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

5.5 Other AUTOSAR modules - dependencies

Development Error Tracer:

This module is necessary for enabling Development error detection. The API function used is `Det_ReportError()`. The activation / deactivation of Development error detection is configurable using ‘`WdgDevErrorDetect`’ configuration parameter.

Diagnostic Event Manager:

This module is necessary for enabling reporting of production relevant error status. The API function used is `Dem_ReportErrorStatus ()`.

Watchdog Interface:

This module is necessary for importing the mode’s type in which the watchdog can be set up. Configuration dependency to other module WDG Driver Integration Manual Module requirements None.

Dependencies

- **GPT** is required to reset periodically the watchdog timeout counter.
- **MCU** module shall be initialized before using GPT .This module is required for setting the global prescaler value and to set the system clock frequency.
- **RESOURCE** is required to select processor derivative. Current Can driver has support for the following derivatives, everyone having attached a Resource file:
`mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64`
`mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100`
`mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100`
`mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144`
`mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64`
`mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208`
`mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100`
`mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bxlq_lqfp144`
`mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208` .
- **ECUM** This module is used for processing the wakeup notifications of GPT.
- **DET** is required for signaling the development error detection (parameters out of range, null pointers, etc).
- **DEM** is required for signaling the production error detection (hardware failure, etc).

Resource Parameters Configuration
`mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100`
`mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100`
`mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64`
`mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64`
`mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100`
`mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64`
`mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144`
`mpc5605bxlu_lqfp176 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176`
`mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208`

Chapter 6

Main API Requirements

6.1 Main functions calls within BSW scheduler

None.

6.2 API Requirements

None

6.3 Calls to Notification Functions, Callbacks, Callouts

6.3.1 Call-back Notifications:

6.3.1 User Notification:

In case GPT callbacks are used for servicing the hardware watchdog, the Wdg module shall implement one or more callback.

The notifications can be configured as pointers to user defined functions. If notification is not desired,

‘NULL_PTR’ shall be configured.

An example of the syntax of this function is as follows:

```
void Wdg_Cbk_GptNotification<x>
```

```
(
```

```
void
```

```
)
```

An extern declaration of this function is available in Wdg_43_Instance<x>_PBcfg.c, Wdg_43_Instance<x>_Lcfg.c, Wdg_43_Instance<x>_Cfg.c. The function has to be implemented by the user.

6.3.3 User Notification:

The WDG Driver provides a notification that is called whenever the defined time period is over. The notifications can be configured as pointers to user defined functions. If notification is not desired, NULL_PTR shall be configured.

An example of the syntax of this function is as follows:

```
void Wdg_Notification
```

```
(
```

```
void
```

```
)
```

The function has to be implemented by the user.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Config data:

```
#ifndef WDG_START_SEC_CONFIG_DATA_<ALIGNMENT>
#define WDG_START_SEC_CONFIG_DATA_<ALIGNMENT>
#define MEMMAP_ERROR
/*Memory Section for Post Build Data to be defined here. Example given in the
next line*/
#pragma ghs section rodata=".mcal_const_cfg"
#endif

#ifndef WDG_STOP_SEC_CONFIG_DATA_<ALIGNMENT>
#define WDG_STOP_SEC_CONFIG_DATA_<ALIGNMENT>
#define MEMMAP_ERROR
/*End of section to be mentioned here. Example given in the next line.*/
#pragma ghs section
#endif
```

For Code:

In case of the code is in Flash:

```
#ifndef WDG_START_SEC_CODE
#define WDG_START_SEC_CODE
#define MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#pragma ghs section text=".mcal_text"
#endif

#ifndef WDG_STOP_SEC_CODE
#define WDG_STOP_SEC_CODE
#define MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

In case of the code is in RAM instead of Flash memory:

```
#ifdef WDG_START_SEC_RAMCODE
#undef WDG_START_SEC_RAMCODE
#undef MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#pragma ghs section text=".ramcode"
#pragma ghs inlineprologue
#endif

#ifdef WDG_STOP_SEC_RAMCODE
#undef WDG_STOP_SEC_RAMCODE
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#pragma ghs nolineprologue
#endif
```

For Variables:

For variables that are never cleared and never initialized by start-up code:

```
#ifdef WDG_START_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef WDG_START_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section bss=".mcal_bss"
#endif

#ifdef WDG_STOP_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef WDG_STOP_SEC_VAR_NO_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

For variables that have been initialized:

```
#ifdef WDG_START_SEC_VAR_INIT_<ALIGNMENT>
#undef WDG_START_SEC_VAR_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section data=".mcal_data"
#endif

#ifdef WDG_STOP_SEC_VAR_INIT_<ALIGNMENT>
#undef WDG_STOP_SEC_VAR_INIT_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif
```

For Constant data:

```
#ifdef WDG_START_SEC_CONST_<ALIGNMENT>
#undef WDG_START_SEC_CONST_<ALIGNMENT>
#undef MEMMAP_ERROR
/*Memory Section for Constants to be defined here.*/
```

```

#pragma ghs section rodata=".mcal_const"
#endif

#ifdef WDG_STOP_SEC_CONST_<ALIGNMENT>
#undef WDG_STOP_SEC_CONST_<ALIGNMENT>
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#pragma ghs section
#endif

```

Tables describe value range of shortcut **ALIGNMENT**:

Table 7-1. Range of <ALIGNMENT>

<ALIGNMENT>	Description
BOOLEAN	Used for variables and constants of size 1 bit
8	used for variables and constants which have to be aligned to 8 bit. For instance used for variables of size 8 bit or used for composite data types: arrays, structs and unions containing elements of maximum 8 bits.
16	used for variables and constants which have to be aligned to 16 bit. For instance used for variables of size 16 bit or used for composite data types: arrays, structs and unions containing elements of maximum 16 bits.
32	used for variables and constants which have to be aligned to 32 bit. For instance used for variables of size 32 bit or used for composite data types: arrays, structs and unions containing elements of maximum 32 bits.
UNSPECIFIED	used for variables, constants, structure, array and unions when SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. For instance used for variables of unknown size.

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration parameters considerations

Configuration parameter class for Autosar WDG driver fall into the following variants as defined below:

8.1 Configuration parameters considerations

Configuration parameter class for Autosar WDG driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
WdgExternalConfiguration			
	WdgExternalContainerRef	VariantPC or VariantLT or VariantPB	N/A
NonAutosar	WdgDisableDemReportErrorStatus	Pre Compile parameter for all Variants of configuration	Pre Compile
WdgGeneral			
	WdgDevErrorDetect	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgDisableAllowed	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgMasterAccessProtection	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgCallbackNotification0	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgIndex	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgTriggerLocation	Pre Compile parameter for all Variants of Configuration	Pre Compile
	WdgVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
WdgDemEventParameterRefs	WDG_E_DISABLE_REJECTED	VariantPC or VariantPB	Post Build
WdgDemEventParameterRefs	WDG_E_MODE_FAILED	VariantPC or VariantPB	Post Build

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
WdgDemEventParameterRefs	WDG_E_MISS_TRIGGER	VariantPC or VariantPB	Post Build
WdgModeConfig			
	WdgInstance	VariantPC VariantLT VariantPB	Pre Compile / Link Time / Post Build
	WdgDefaultMode	VariantPC or VariantPB	Pre Compile / Post Build
	WdgKeyedService	VariantPC	Pre Compile
	WdgKeyedServiceValue	VariantPC	Pre Compile
	WdgSettingsFast	VariantPC or VariantPB	Pre Compile / Post Build
	WdgSettingsOff	VariantPC or VariantPB	Post Build / Post Build
	WdgSettingsSlow	VariantPC or VariantPB	Post Build/ Post Build
WdgSettingsFast			
	WdgReadOnly	VariantPC or VariantPB	Pre Compile / Post Build
	WdgHardLock	VariantPC or VariantPB	Pre Compile / Post Build
	WdgWindowMode	VariantPC or VariantPB	Pre Compile / Post Build
	WdgRunsInStop	VariantPC or VariantPB	Pre Compile / Post Build
	WdgRunsInDebug	VariantPC or VariantPB	Pre Compile / Post Build
	WdgClockSelection	VariantPC or VariantPB	Pre Compile / Post Build
	WdgOperationMode	VariantPC or VariantPB	Pre Compile / Post Build
	WdgResetOnInvalidAccess	VariantPC or VariantPB	Pre Compile / Post Build
	WdgTimeoutPeriod	VariantPC or VariantPB	Pre Compile / Post Build
	WdgWindowPeriod	VariantPC or VariantPB	Pre Compile / Post Build
WdgSettingsOff			
	WdgReadOnly	VariantPC or VariantPB	Pre Compile / Post Build
	WdgHardLock	VariantPC or VariantPB	Pre Compile / Post Build
WdgSettingsSlow		VariantPC or VariantPB	Pre Compile / Post Build
	WdgReadOnly	VariantPC or VariantPB	Pre Compile / Post Build
	WdgHardLock	VariantPC or VariantPB	Pre Compile / Post Build

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	WdgWindowMode	VariantPC or VariantPB	Pre Compile / Post Build
	WdgRunsInStop	VariantPC or VariantPB	Pre Compile / Post Build
	WdgRunsInDebug	VariantPC or VariantPB	Pre Compile / Post Build
	WdgClockSelection	VariantPC or VariantPB	Pre Compile Post Build
	WdgOperationMode	VariantPC or VariantPB	Pre Compile Post Build
	WdgResetOnInvalidAccess	VariantPC or VariantPB	Pre Compile Post Build
	WdgTimeoutPeriod	VariantPC or VariantPB	Pre Compile Post Build
	WdgWindowPeriod	VariantPC or VariantPB	Pre Compile Post Build
	WdgDefaultMode	VariantPC or VariantPB	Pre Compile Post Build

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating Watchdog :

- Generate the required WDG configurations. For more details refer to section [Files required for Compilation](#)
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Sections to be defined in MemMap.h](#)
- Make sure all include files for compilation are as per section [ISR Reference](#)
- Map the ISRs to their vector locations. For more details refer to section [ISR to configure within OS – dependencies](#)
- Compile & build the WDG with all the dependent modules. For more details refer to section [Building the Driver & Other AUTOSAR modules - dependencies](#)



Chapter 10

ISR Reference

ISR functions exported by the WDG driver.

10.1 Software specification

The following sections contains driver software specifications.

10.1.1 Define Reference

Constants supported by the WDG driver.

10.1.2 Macros Reference

Macros supported by the WDG driver.

10.1.3 Enum Reference

Enumeration of all constants supported by the WDG driver.

10.1.4 Function Reference

Functions supported by the WDG driver.

10.1.4.1 Function Wdg_Isr

This function treats the SWT interrupt.

Details:

Interrupt Service Routine Addition.

Pre: This ISR is only required if the WDG_ISR_USED option is enabled.

Prototype: `void Wdg_Isr(void);`

10.1.5 Structs Reference

Data structures supported by the WDG driver.

10.1.6 Types Reference

Types structures supported by the WDG driver.

10.1.7 Variables Reference

Variables supported by the WDG driver.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13WDGASR4.0 Rev0003R1.0.1
Revision 1.2